Introduction to Octave

Michelle Shaw, CRC ACEnet
November 26, 2010

To Cover...

- basic topics:
  - what octave is/isn't
  - starting, stopping, getting help from octave
  - assigning and using scalar variables
  - math operators and "scientific calculator" functions
- intermediate:
  - vectors and matrices
  - matrix operations and commonly used functions
  - basic plotting of functions
- advanced - scripting:
  - loops
  - functions
  - I/O

What Octave Is and Isn't...

- what octave is:
  - prototyping language built for mathematical modeling
  - matlab clone
  - combination of math functions and a plotting utility
- what octave isn't:
  - a replacement for more general programming and scripting languages
  - a high-speed mathematical modeler
  - a program for doing algebraic manipulation (like Matlab and Maple are capable of doing
- NOTE: octave is designed primarily to do matrix computations and is not a one-stop shopping for mathematical tools

Basic Topics

Starting, Stopping and Help Files

- can use octave on Linux, Windows and Mac (using fink)
- NOTE: it requires GNUplot to run as this is the utility it uses for plotting
- to start:

```
dshaw@placentia:althead ~ > octave
GNU Octave, version 2.1.57 (x86_64-unknown-linux-gnu).
                                ...
octave:1>
```

- to quit:

```
octave:1> quit
```

Starting, Stopping and Help Files

- to get command line help, use the help command
- several ways to do this:
  - help: lists topics that you can get help on
  - help <command>: tells you how to use <command>
  - help -i <topic>: does a search for <topic>
  - help -i help: opens the help pages
- help pages are like the linux info pages:
  - they can be searched using /<keyword> like vi
  - to exit, type q
  - n, p: next, previous
  - <space>: scroll ahead a page
  - up, down arrows: navigate by line up and down

Starting, Stopping and Help Files

- some examples:

```
octave:5> help
Help is available for the topics listed below.
                ...
octave:7> help sin
sin is a built-in mapper function
                ...
-- Mapping Function:  sin (X)
     Compute the sin of each element of X.
octave:8> help -i atan
                ...
 -- Mapping Function:  atan (X)
     Compute the inverse tangent of each element of X.
                ...
```

Notes on Octave Format and Behavior

- octave has some interesting behavior, some quite similar to matlab and some similar to other common linux utilities
  - up and down arrows to navigate command history
  - tab completion for commands
  - manuals have same navigation interface as the linux info pages
  - you don't need a line "ender" like perl and C's semicolon, but...
    - not adding a semicolon causes octave to echo back results of a computation or assignment
    - adding a semicolon turns this behavior off and nothing is echoed

Assigning and Using Scalar Variables

- "scalar" variables: basically single-value variables
- like python, they don't need to be assigned a type
- assign and use them in a similar manner:
  - <variable> = <value> and access using <variable>
  - examples:

```
octave:11> x = 5
x = 5
octave:12> b = 7;
octave:13>
octave:14> printf("%d \n", x)
5
octave:16> string = "mystring"
string = mystring
octave:17> printf("%s\n", string)
mystring
octave:18> x*b
ans = 35
```

Operators

- the usual suspects:
  - arithmetic: +, -, *, /, **
  - binary: &, |, !
  - comparison (numeric): >, >=, <, <=, ==, !=
  - strings: strcmp function
- the not-so-usual suspects:
  - arithmetic:
    - \ : left division (e.g. b\x is equivalent to x/b)
    - -<var>: negation
    - ^ : equivalent to **
  - comparison:
    - ~=, <> : both equivalent to !=
  - boolean:
    - ~ : equivalent to !
    - &&, || : equivalent to (...) && (...) and (...) || (...)

Some Common Math Functions

- octave has several simple and common math functions
- some examples:
  - trig. functions: sin, cos, tan, sec, csc, cot, plus arc and hyperbolic versions of these
  - rounding: ceil, floor, rem, round, mod
  - logarithmic: log, log10, log2
  - others: sqrt, max,
- examples:

```
octave:42> x = 1.2;
octave:52> sin(x)
ans = 0.93204
octave:53> cot(x)
ans = 0.38878
octave:54> floor(x)
ans = 1
octave:56> log10(x)
ans = 0.079181
```

Intermediate Topics

Matrices and Vectors

- common variable that is more than one dimension is the matrix
- the 1D version is the vector
- that is, operations in octave operate on matrices, not arrays
- assigning matrices: several ways....
- simple syntax:
  - <matrixname> = [<values>]
  - separate elements by a comma or space
  - separate rows by a semicolon
  - example:

```
octave:60> X = [1, 2, 3]
X =
   1  2  3
octave:61> Y = [4, 5; 6, 7]
Y =
   4  5
   6  7
```

Matrices and Vectors

- colon notation syntax:
  - <vectorname> = <1st value>:<step>:<last value>
    - <1st value> is the first value in the vector
    - <last value> is the last value in the vector
    - <step> is the vector value's increment
  - <matrixname> = [<1st>:<step>:<last>;<1st>:<step>:<last>...]
  - some examples:

```
octave:66> B = 7:2:14
B =
   7   9  11  13
octave:67> C = [1:2:10;3:3:15]
C =
   1   3   5   7   9
   3   6   9  12  15
```

Matrices and Vectors

- vectors and matrices from other vectors and matrices:
  - <matrixname> = [<vector1>;<vector2>;...]
    - NOTE: row, col numbers must match, otherwise you will get an error message
  - examples:

```
octave:68> D = [1, 3, 5, 7]
D =
   1   3   5   7
octave:69> E = [2, 4, 6]
E =
   2   4   6
octave:70> F = [D; E]
error: number of columns must match (3 != 4)
error: evaluating assignment expression near line 70, column 3
octave:70> G = [D; E, 8]
G =
   1   3   5   7
   2   4   6   8
```

Matrices and Vectors

- accessing matrix and vector elements can be done similarly to assignment:
  - specify an element by row, column pair:

```
octave:71> G(1,3)
ans = 5
```

  - specify a row or column using ":" as a "wildcard":

```
octave:72> G(:,3)
ans =
   5
   6
```

  - specify "slices" using the colon notation:

```
octave:74> G(1:2,3:4)
ans =
   5   7
   6   8
```

Matrix and Vector Operations

- operators for matrices and vectors work a little differently...
  - some work as expected:
    - +, -, *, / : matrix and vector addition, subtraction, multiplication and division
  - some work element-by-element:
    - .+, .-, .*, ./ : add, subtract, multiply and divide
  - the difference:

```
octave:75> A
A =
   2  5  8
octave:79> E
E =
   2  4  6
octave:80> A*(E')
ans = 72
octave:82> A.*E
ans =
   4  20  48
```

Matrix and Vector Operations

- some other useful operations:
  - ' : the transpose of a matrix or vector:

```
octave:88> E'
ans =
   2
   4
   6
```

  - max, min: maximum, minimum value of a vector:

```
octave:89> max(E)
ans = 6
octave:90> min(E)
ans = 2
```

  - boolean: any, all (return 1 if any/all of vector E are nonzero)

```
octave:97> any(E)
ans = 1
octave:98> all(E)
ans = 1
```

Matrix and Vector Operations

- adding/deleting rows and columns:
  - rows/cols that are assigned are added if they don't exist
  - assigning an empty array to a row/col deletes it

```
octave:102> G(2,:) = []
G =
   1  3  5  7
octave:103> G(2,:) = [2, 3, 5, 7]
G =
   1  3  5  7
   2  3  5  7
```

- checking size:
  - rows(A)
  - columns(A)

```
octave:104> rows(G)
ans = 2
octave:105> columns(G)
ans = 4
```

Matrix and Vector Operations

- size(<matrix>, <row or col number>) : number of rows and columns
- length(<matrix>) : size of row or column, whichever is larger
- numel(<matrix>) : number of elements

```
octave:107> length(G)
ans = 4
octave:108> numel(G)
ans = 8
```

- other useful operations:
  - inv(<matrix>) : inverse of a matrix
  - det(<matrix>) : determinant of a matrix
  - ... and so many others ...

Special Matrices

- ones(<M>, <N>) : create a matrix of ones, size <M> by <N>
- zeros(<M>, <N>) : create a matrix of zeros, size <M> by <N>
- eye(<no. elements>) : identity matrix with <no. elements>
- diag(<diagonal elements>) : diagonal matrix with elements specified as argument

```
octave:109> ones(2, 2)
ans =
   1  1
   1  1
octave:111> eye(2)
ans =
   1  0
   0  1
octave:112> diag([9 5])
ans =
   9  0
   0  5
```

Plotting

- octave is capable of plotting in 2D and 3D
- plotting GUI not native to octave:
  - uses GNUplot to display the plot
- can plot the results of an octave calculation or a dataset
- several 2D and 3D plotting functions
- the most common 2D plot function is called "plot":
  - can take several arguments
  - simplest format is: plot (<vector>)
    - <vector> is the thing being plotted, values are the Y coordinates of the plot
    - X coordinates are the indices of <vector>
  - more generic format: plot(<X>, <Y>):
    - <X> and <Y> are the X and Y coordinates, respectively
    - <X> can also be a function and <Y> its values

Plotting

```
octave:4> A = [1, 3, 5, 7, 11];
octave:5> plot(A)
octave:10> B = [0:pi/16:2*pi];
octave:11> plot(B, sin(B))
```

- multiple plots can be put on the same graph:

```
octave:27> E = [-1:0.1:1];
octave:28> plot(E, acos(E))
octave:29> plot(E, acos(E), ':', E, asin(E))
```

- axis and plot formatting is also possible using a small function set:
  - xlabel('<label>'), ylabel('<label>'), zlabel('<label>'): add axis labels, <label> is the string label for that axis
  - title('<title>') : add a title <title>
  - grid [on/off] : adds/removes grid
  - plot(<X>, <Y>, <format>) : <format> specifies line/point format

More Advanced - Scripting

Control Statements - if

- if statement syntax:

```
if <condition>
  do something
elseif <condition>
  do something else
else
  the default action
end
```

- example:

```
octave:62> x = 2;
octave:63> y = 7;
octave:64> if x > y
> x
> else
> y
> end
y = 7
```

Control Statements - Switch/Case

- syntax of the switch/case:

```
switch <expression>
    case <selection1>
        do something
    ...
 endswitch
```

- example:

```
octave:71> b = 1;
octave:72> switch (b)
> case 0
>   disp('b is false')
> case 1
>   disp('b is true')
> otherwise
>   disp('bad value for b')
> endswitch
b is true
```

Loops - for

- for loop syntax:

  for <var>=<list>

  do something

  end

- example:

```
octave:73> for i = 1:1:5
> i
> end
i = 1
i = 2
i = 3
i = 4
i = 5
```

Loops - while

- while loop syntax:

```
while <condition>
    do something
end
```

- example:

```
octave:74> x = 7;
octave:75> y = 0;
octave:76> while y < x
> y++
> end
ans = 0
ans = 1
ans = 2
ans = 3
ans = 4
ans = 5
ans = 6
```

Functions

- octave has the functionality for users to create their own functions
- useful if you want to wrap up a multi-step procedure you use often
- these functions can be created, then stored in internal files:
  - NOTE: for the matlab'ers among us, the file format is familiar - fnname.m
- syntax:

```
function <name> (<arguments>)
    the guts of the function
endfunction
```

- to use the function, you call it like any other, e.g.:
  - x = myfunc(arg1)
  - myfunc2

Functions - Hello World Example

- simplest function doesn't take arguments and doesn't return anything:

```
octave:78> function hello
> printf("hello world! \n")
> endfunction
```

- we call this function using:

```
octave:79> hello
hello world!
```

- more complex things next week...

More Information

- I/O next week...
- help files are quite detailed, but there's lots of online tutorials
- ask me, though I've not worked with octave/matlab in a few years...
- plenty of online documentation
- practice using the interpreter - it is quite handy, much like python